Zero to Hero Study Handbook: qlora-complaint-intent-classifier

Module 1: Foundations & Architecture

1) High-level overview

This repository builds a banking intent classifier using QLoRA fine-tuning on `Qwen/Qwen2.5-1.5B-Instruct` for the `PolyAI/banking77` dataset, then serves predictions through a CLI inference script.

Primary use cases implemented in this repo:

- Train and evaluate a 77-class intent classifier in a notebook workflow.
- Compare base-model zero-shot behavior vs fine-tuned behavior.
- Save LoRA adapter artifacts for reuse (`qlora-banking77-adapter/`).
- Run inference in single-query, batch, or interactive CLI mode (`inference.py`).

Primary evidence files:

- `README.md`
- `pyproject.toml`
- `make_notebook.py`
- `qlora_complaint_intent_classifier.ipynb`
- `inference.py`
- `qlora-banking77-adapter/adapter_config.json`

2) Core paradigms and patterns used here

1. Procedural scripting Definition: Logic is organized as sequential script steps (top-to-bottom execution) rather than deep class hierarchies. Where used:

- `inference.py` (`main()`, `load_model()`, `predict()`).
- `make_notebook.py` (sequential construction of notebook cells).

1. Notebook-centered ML pipeline Definition: Data loading, preprocessing, baseline, fine-tuning, and analysis are executed in staged notebook cells. Where used:

- `qlora_complaint_intent_classifier.ipynb` (generated by `make_notebook.py`).

1. Adapter-based fine-tuning (QLoRA) Definition: Base model remains frozen in 4-bit; small low-rank adapters are trained and saved. Where used:

- Notebook code cells defining `BitsAndBytesConfig`, `LoraConfig`, `SFTConfig`, `SFTTrainer`.

- Saved adapter metadata in `qlora-banking77-adapter/adapter_config.json`.

1. Prompt-as-classifier pattern Definition: Classification is framed as constrained text generation with a strict system instruction. Where used:

- `SYSTEM_MSG` in both notebook and `inference.py`.
- `tokenizer.apply_chat_template(...)` + greedy decoding (`do_sample=False`).

1. Rule-based post-generation normalization Definition: Generated label strings are normalized and mapped through deterministic fallback rules. Where used:

- `_normalise()` and `predict()` in `inference.py`.
- `predict_intent()` in notebook code cells.

3) Architecture: key components and interactions

Core components:

1. Environment and dependency contract

- `pyproject.toml` declares runtime dependencies and `[tool.uv] torch-backend = "cu128"`.
- `uv.lock` pins transitive versions.
- `.python-version` pins Python `3.12.10`.

1. Notebook source generator

- `make_notebook.py` programmatically builds the notebook JSON (`md()`, `code()`, `_uid()`).

1. Main training/evaluation runtime

- `qlora_complaint_intent_classifier.ipynb` contains the operational ML flow.

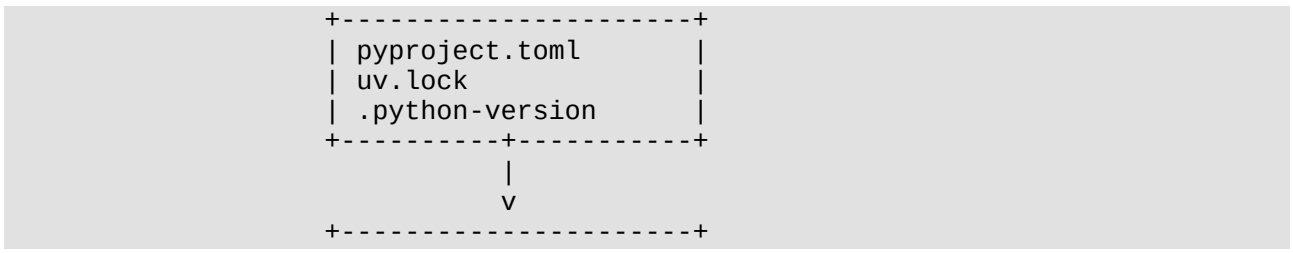
1. Saved model artifacts

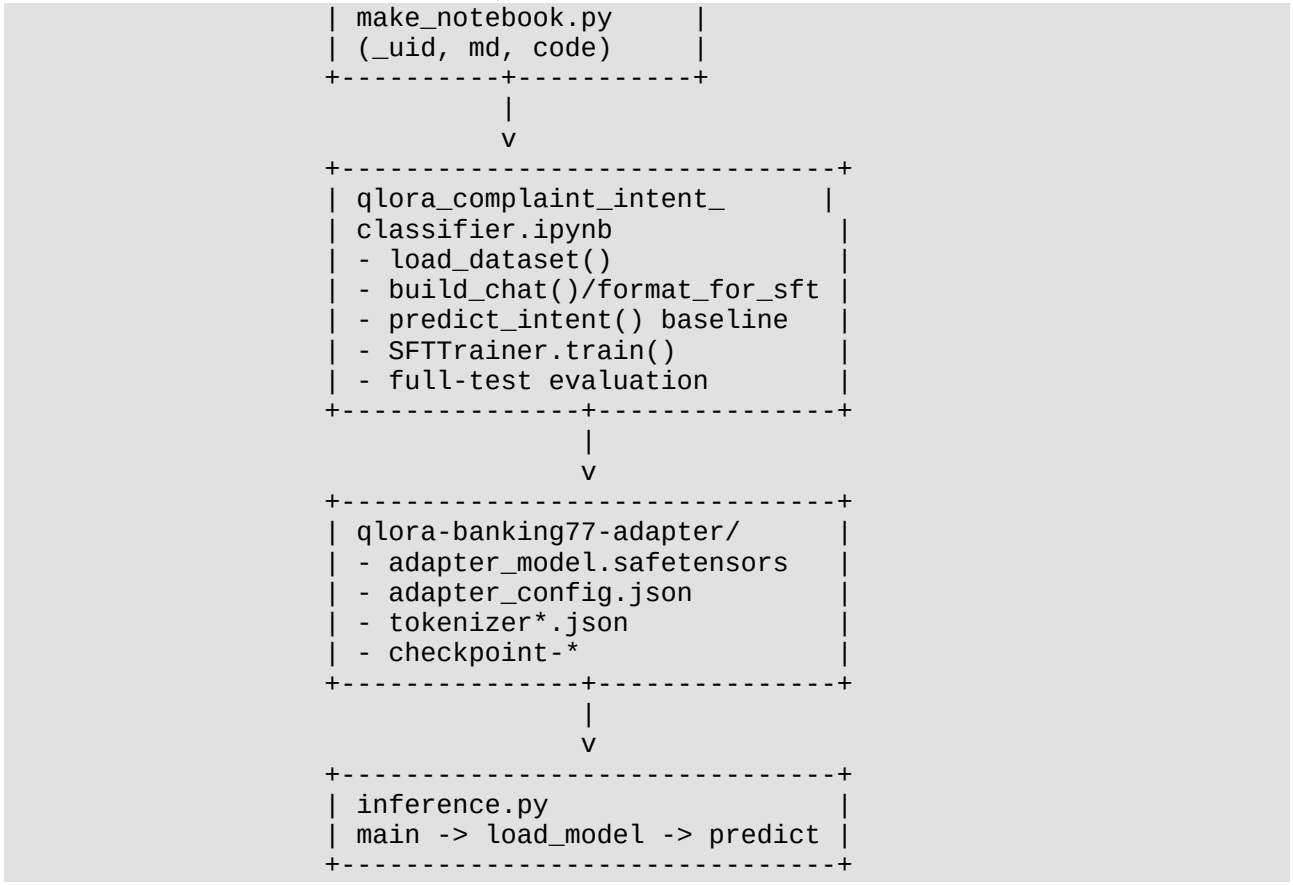
- `qlora-banking77-adapter/` stores adapter weights, tokenizer artifacts, and checkpoints.

1. Inference runtime

- `inference.py` loads base model + adapter and predicts intent labels for queries.

Main flow diagram:





Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`README.md`	Project overview, setup, run commands, training summary	N/A	Runtime commands, hardware expectations, training summary table
`pyproject.toml`	Python project/dependency manifest for `uv`	N/A	`requires-python = ">=3.12.10"`, dependency list, `[tool.uv].torch-backend = "cu128"`
`.python-version`	Python interpreter pin	N/A	`3.12.10`
`uv.lock`	Locked dependency graph	N/A	Pinned resolved package versions
`make_notebook.py`	Generates `qlora_complaint_intent_classifier.ipynb` from code/markdown cell strings	`_uid()`, `md()`, `code()`	Notebook metadata (`nbformat`, kernel info), cell source content
`qlora_complaint_intent_classif`	Main training + evaluation runtime	`build_chat()`, `format_for_sft(`	`SEED`, `MODEL_ID`, `MAX_LENGTH`,

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
ier.ipynb	notebook	), predict_intent( ) (defined in code cells)	SYSTEM_MSG, OUTPUT_DIR, BASELINE_N
inference.py	CLI inference entry point using saved adapter	load_model(), _normalise(), predict(), main()	BASE_MODEL_ID, ADAPTER_DIR, MAX_NEW_TOKENS, DEVICE, SYSTEM_MSG, LABEL_NAMES
qlora-banking77-adapter/ adapter_config.json	Persisted LoRA adapter hyperparameters and target modules	N/A	peft_type, r, lora_alpha, lora_dropout, target_modules, task_type, base_model_name_or_path
qlora-banking77-adapter/ adapter_model.safetensors	Saved LoRA adapter weights	N/A	Tensor weights for configured target modules
qlora-banking77-adapter/ tokenizer_config.json	Tokenizer runtime metadata used with adapter	N/A	tokenizer_class, model_max_length, eos_token, pad_token, extra_special_tokens
qlora-banking77-adapter/ chat_template.jinja	Prompt serialization template used by tokenizer	Jinja template logic	Role framing with <|im_start|>/<|im_end|> and generation prompt behavior
qlora-banking77-adapter/ checkpoint-626/	Epoch-1 checkpoint state	N/A	trainer_state.json (global_step: 626, epoch: 1.0)
qlora-banking77-adapter/ checkpoint-1252/	Epoch-2 checkpoint state	N/A	trainer_state.json (global_step: 1252, epoch: 2.0)
qlora-banking77-adapter/	Epoch-3/final checkpoint state	N/A	trainer_state.json (global_step: 1878, epoch: 3.0)

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`checkpoint-1878/`			
`label_distribution.png`	Notebook-generated visualization of train label distribution	N/A	Saved plot artifact
`query_length.png`	Notebook-generated visualization of query lengths	N/A	Saved plot artifact
`base_vs_finetuned.png`	Notebook-generated metric comparison plot	N/A	Saved plot artifact
`confusion_matrix.png`	Notebook-generated confusion matrix (most-confused intents)	N/A	Saved plot artifact

Module 3: Core Execution Flows

Flow A: Notebook generation flow (`make_notebook.py` -> `.ipynb`)

Entrypoint:

- Script execution of `make_notebook.py`.

Step-by-step:

1. `_uid()` generates UUID cell IDs.
2. `md(src)` converts markdown text to a notebook markdown-cell dictionary.
3. `code(src)` converts Python source text to a notebook code-cell dictionary.
4. `cells.append(md(...))` and `cells.append(code(...))` build the full notebook content.
5. A final `notebook` dict is assembled with `nbformat`, `metadata`, and `cells`.
6. `json.dump(...)` writes `qlora_complaint_intent_classifier.ipynb`.

Short real code fragment:

```
def md(src: str):
    ...
    return {"cell_type": "markdown", "id": _uid(), "metadata": {}, "source":
source}

def code(src: str):
    ...
    return {
        "cell_type": "code",
        "execution_count": None,
        "id": _uid(),
        "metadata": {},
```

```
        "outputs": [],
        "source": source,
    }
```

Input/output shape:

- Input: Python strings inside `make_notebook.py`.
- Output: Notebook JSON object with keys:

```
{
  "nbformat": 4,
  "nbformat_minor": 5,
  "metadata": {...},
  "cells": [...]
}
```

Flow B: Training and evaluation flow in `qlora_complaint_intent_classifier.ipynb`

Entrypoint:

- Running notebook cells sequentially.

Step-by-step:

1. Environment and reproducibility setup

- Imports ML stack.
- Sets `SEED = 42` for `random`, `numpy`, and `torch`.

1. Dataset loading

- `raw_ds = load_dataset("PolyAI/banking77", revision="refs/convert/parquet")`
- `train_ds = raw_ds["train"]`, `test_ds = raw_ds["test"]`
- `LABEL_NAMES = [n.lower() for n in train_ds.features["label"].names]`

1. Prompt/data formatting

- `SYSTEM_MSG` defines strict label-only response behavior.
- `build_chat(text, intent=None)` builds chat-formatted prompts.
- `format_for_sft(example)` returns training rows in a single-field text format.

1. Baseline evaluation (base model, zero-shot)

- Loads quantized base model using `BitsAndBytesConfig`.
- Uses `predict_intent(text, model)` with greedy decoding and normalization fallback.
- Evaluates sampled test subset (`BASELINE_N = 200`).

1. QLoRA fine-tuning

- Loads base model in 4-bit.
- Applies `LoraConfig(...)` with target modules `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`.
- Defines `SFTConfig(...)` (`num_train_epochs=3`, `per_device_train_batch_size=4`, `gradient_accumulation_steps=4`, `optim="paged_adamw_8bit"`, `learning_rate=2e-4`, `bf16=True`).
- Trains with `SFTTrainer(...).train()`.

1. Save adapter/tokenizer artifacts

- `trainer.save_model(OUTPUT_DIR)`
- `tokenizer.save_pretrained(OUTPUT_DIR)` where `OUTPUT_DIR = "./qlora-banking77-adapter"`

1. Full test evaluation and analysis

- Reloads base model and attaches adapter via `PeftModel.from_pretrained(_base, OUTPUT_DIR)`.
- Evaluates full `test_ds` (3,080 rows) to compute accuracy and macro F1.
- Produces comparison table and plots (`base_vs_finetuned.png`, `confusion_matrix.png`).

Short real code fragment:

```
def format_for_sft(example):
    return {"text": build_chat(example["text"], LABEL_NAMES[example["label"]])}

training_args = SFTConfig(
    output_dir=OUTPUT_DIR,
    dataset_text_field="text",
    max_length=MAX_LENGTH,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    optim="paged_adamw_8bit",
    learning_rate=2e-4,
    bf16=True,
)
```

Key input/output shapes:

```
# raw Banking77 row
{"text": <str>, "label": <int>}

# SFT-formatted row
{"text": <chat_prompt_str>}

# chat message item
{"role": "system" | "user" | "assistant", "content": <str>}
```

Stored notebook-output evidence (static read, not rerun):

- Dataset rows: train `10,003`, test `3,080`.

- Baseline metrics (200 examples): accuracy `4.50%`, macro F1 `4.18%`.
- Fine-tuned metrics (3,080 examples): accuracy `92.37%`, macro F1 `91.97%`.
- Training summary: `global_step = 1878`, `training_loss = 0.4705`, runtime `43.2 min`.

Flow C: Runtime inference flow (`inference.py`)

Entrypoint:

- `if __name__ == "__main__": main()`

Step-by-step:

1. Parse CLI args in `main()`:

- positional `query` (optional)
- `--batch FILE`
- `--adapter PATH` (default `./qlora-banking77-adapter`)

1. Adapter existence check

- Exits with code `1` if adapter path does not exist.

1. `load_model(adapter_dir)`

- Creates 4-bit quantization config (`BitsAndBytesConfig`).
- Loads base model (`AutoModelForCausalLM.from_pretrained(...)`).
- Attaches adapter (`PeftModel.from_pretrained(...)`).
- Loads tokenizer from adapter path.

1. `predict(text, model, tokenizer)`

- Builds message list with `SYSTEM_MSG` and user query.
- Serializes prompt via `tokenizer.apply_chat_template(..., add_generation_prompt=True)`.
- Tokenizes to max length `256` and runs greedy generation (`do_sample=False`, `max_new_tokens=16`).
- Normalizes generated text and maps to known labels via exact match -> prefix match -> character-overlap fallback.

1. Mode-specific output

- Single query mode prints `Query` and `Intent` lines.
- Batch mode prints tab-separated `label<TAB>query` per line.
- Interactive mode loops on `input()`.

Short real code fragment:

```
messages = [
    {"role": "system", "content": SYSTEM_MSG},
    {"role": "user", "content": text},
]
prompt = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
inputs = tokenizer(prompt, return_tensors="pt", truncation=True,
max_length=256).to(DEVICE)
```

CLI input/output contract:

```
# input (single)
uv run python inference.py "My card was declined"

# output (single)
Query  : My card was declined
Intent : declined_card_payment

# input (batch)
uv run python inference.py --batch queries.txt

# output (batch)
<label>\t<original_query>
```

Important consistency note from static inspection:

- `inference.py` hardcodes `LABEL_NAMES` and includes `"reverted_card_payment?"`.
- Notebook training logic derives labels from dataset metadata (`train_ds.features["label"].names`, lowercased).
- A learner should verify this label-list consistency before treating the CLI as canonical.

Module 4: Setup & Run Guide

1) Clean-machine prerequisites

From `README.md`, `.python-version`, and `pyproject.toml`:

- OS target in docs: Ubuntu 22.04 / 24.04 / 26.04.
- Python: `3.12.10`.
- Package manager: `uv`.
- GPU target in docs: NVIDIA GPU with >= 8 GB VRAM, CUDA 12.x/13.x.

Core dependency families declared in `pyproject.toml`:

- DL/model stack: `torch`, `transformers`, `peft`, `bitsandbytes`, `trl`, `accelerate`.
- Data/eval stack: `datasets`, `evaluate`, `scikit-learn`, `numpy`, `pandas`.
- Visualization/notebook stack: `matplotlib`, `seaborn`, `jupyter`, `ipykernel`, `ipywidgets`, `nbformat`.

2) Typical install sequence

```
git clone https://github.com/pypi-ahmad/qlora-complaint-intent-classifier.git
cd qlora-complaint-intent-classifier
uv sync
uv run python -m ipykernel install --user \
  --name qlora-complaint-intent-classifier \
  --display-name "Python 3.12 (qlora-env)"
```

3) Main ways to run this project

Notebook training/evaluation path:

```
uv run jupyter notebook qlora_complaint_intent_classifier.ipynb
```

Notebook regeneration path (if editing notebook source script):

```
uv run python make_notebook.py
```

Inference CLI path:

```
uv run python inference.py "My card was declined at the supermarket"
uv run python inference.py
uv run python inference.py --batch queries.txt
```

4) Environment variables and config files

Required `.env` keys in this repository:

- None are explicitly required by repository source files.

Optional key observed in notebook output warnings:

- `HF_TOKEN` can improve Hugging Face Hub rate limits; not hard-required in code.

Important config files to understand:

- `pyproject.toml`
- `.python-version`
- `qlora-banking77-adapter/adapter_config.json`
- `qlora-banking77-adapter/tokenizer_config.json`

5) Database/external service setup

Database migrations/seeding:

- No database or migration system is present in this repo.

External services used by runtime path:

- Hugging Face dataset download (`PolyAI/banking77`).
- Hugging Face model download (`Qwen/Qwen2.5-1.5B-Instruct`).

6) PDF export of this handbook

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

1) Ordered study plan for a new learner

1. Read `README.md` to get project scope, expected hardware, and user-facing commands.
2. Read `pyproject.toml` and `.python-version` to understand runtime constraints.
3. Read `inference.py` end-to-end to understand the deploy-time path (`main -> load_model -> predict`).
4. Read `make_notebook.py` to see how the training notebook is assembled and what logic is embedded.
5. Open `qlora_complaint_intent_classifier.ipynb` and map each section to corresponding cell logic.
6. Inspect `qlora-banking77-adapter/adapter_config.json` to connect training choices to persisted adapter settings.
7. Inspect `checkpoint-626`, `checkpoint-1252`, `checkpoint-1878` `trainer_state.json` files to understand epoch/step progression.
8. Re-read `README.md` and note where docs are consistent or inconsistent with notebook outputs.

2) Practice exercises (with solution outlines)

1. Exercise: Identify all operational entrypoints. Task: List every file that a user directly runs and what it does. Solution outline: `inference.py` (CLI prediction), `qlora_complaint_intent_classifier.ipynb` (training/evaluation), `make_notebook.py` (notebook generation).
2. Exercise: Reconstruct the exact inference contract. Task: Extract accepted CLI arguments and all output formats from `inference.py`. Solution outline: `query` positional, `--batch`, `--adapter`; outputs are single-query two-line format, batch tab-separated format, and interactive prompt loop.
3. Exercise: Trace label handling end-to-end. Task: Compare label source in notebook vs label source in `inference.py`. Solution outline: Notebook derives labels dynamically from dataset metadata and lowercases them; `inference.py` uses a hardcoded `LABEL_NAMES` list.
4. Exercise: Rebuild the QLoRA training config from source. Task: Write down all fine-tuning-critical parameters and where they appear. Solution outline: `BitsAndBytesConfig` (4-bit NF4 + double quant + bf16 compute), `LoraConfig` (`r=16`, `alpha=32`, `dropout=0.05`, target modules), `SFTConfig` (`epochs=3`, batch `4`, grad accumulation `4`, `paged_adamw_8bit`, cosine LR scheduler).
5. Exercise: Compute effective batch size and expected steps. Task: From notebook code, compute effective batch size and show formula for steps per epoch. Solution outline: Effective batch = `per_device_train_batch_size *`

`gradient_accumulation_steps = 4 * 4 = 16`; steps per epoch computed as `len(train_sft) // eff_batch`.

6. Exercise: Explain persisted artifact roles. Task: For each file type in `qlora-banking77-adapter/`, explain why it is needed. Solution outline: `adapter_model.safetensors` stores trained adapter weights; `adapter_config.json` stores adapter architecture/hyperparameters; tokenizer files preserve prompt/token behavior; checkpoint folders store optimizer/scheduler/RNG/trainer state.
7. Exercise: Verify metrics provenance without rerunning code. Task: Identify where baseline and fine-tuned metrics came from. Solution outline: They are present in stored notebook outputs in cells printing baseline (`BASELINE_N=200`) and full-test fine-tuned evaluation (`len(test_ds)=3,080`).
8. Exercise: Find one documentation inconsistency. Task: Compare README dataset counts with stored notebook output. Solution outline: README table says train split `13,083`; notebook output shows train split `10,003` for `revision="refs/convert/parquet"`.
9. Exercise: Explain normalization fallback logic in `predict()`. Task: Describe each fallback stage and why it exists. Solution outline: Normalize generated string -> exact label match -> prefix-based match -> character-overlap heuristic to avoid failure on punctuation/noisy outputs.
10. Exercise: Design a safe refactor to reduce train/inference drift. Task: Propose a minimal change that keeps behavior but removes hardcoded label drift risk. Solution outline: Persist canonical label list during training to a JSON artifact and load that file in `inference.py` instead of a hardcoded list.

3) Learner self-verification checklist

- Can you explain the complete flow from notebook generation (`make_notebook.py`) to trained artifacts (`qlora-banking77-adapter/`)?
- Can you explain exactly how `inference.py` maps a raw query string to one final intent label?
- Can you name the critical QLoRA settings and where each is defined?
- Can you describe the data shape at each stage (`raw_ds` row, `format_for_sft` row, prediction return)?
- Can you distinguish baseline evaluation scope (`200` examples) vs fine-tuned evaluation scope (`3,080` examples)?
- Can you explain what each checkpoint directory (`626`, `1252`, `1878`) represents?
- Can you identify at least one potential train/inference consistency risk in current code?
- Can you bootstrap this project on a clean machine using only repo docs and manifests, without guessing missing config?